

0.56.3. La fonction losange() — tourner en losange

`#include <losange>` : `losange(x, y, tuile, taille, sens, vitesse, tours)`, le carré posé sur la pointe.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `losange(...)` fait tourner la tuile en **losange** autour de (x, y) : un carré posé sur la pointe, ses côtés en diagonale. Les pointes sont à `taille` cases du centre.

La ligne à écrire : `#include <losange>`.

Ses arguments : les mêmes sept que `carre()`, dans le même ordre.

Ce qu'elle coûte : environ **510 octets**, plus la police entière.

Essaie : une taille de 4.

On retrouve losange dans 10 leçons

[0.57](#), [0.57.1](#), [0.57.2](#), [0.62](#), [0.62.1](#), [0.62.2](#), [0.63](#), [0.63.2](#), [0.64](#), [0.64.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1358 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
losange()      449 o ← main() – une fonction fournie par le langage
main()         38 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 4    52 o sur place, dans losange() + AdresseCase (10 o)
effacer() × 3  45 o sur place, dans losange() + AdresseCase (10 o), EffacerCases (12 o)
image() × 4    12 o sur place, dans losange(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
losange() × 1  31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

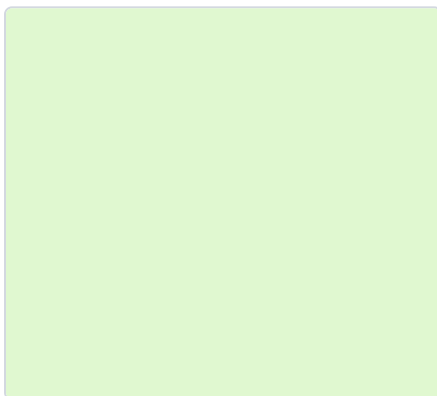
```
// ---- #include <losange> : tourner en losange ----  
  
#include <losange>    // losange() : des tours en losange  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    losange(10, 8, ALPHABET[0], 2, 1, 150, 1);    // pointes à 2 cases du centre  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



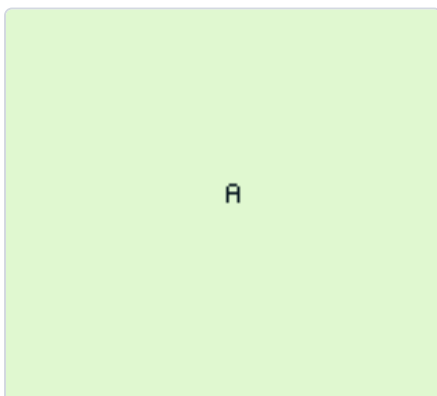
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui fait un tour en losange autour du milieu.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A touche les quatre pointes (10,6 12,8 10,10 8,8)

✓ il revient au centre

Ensuite : **0.57. Un carré sur la pointe : losange** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++